

CS 236: Operating Systems Lab

Spring 2025-26

Lab 8

threads are light, threads are might why oh why are they sometimes impolite?

xv6 is a teaching operating system (developed at MIT) — an OS meant for teaching design and implementation concepts of an OS.

To install xv6 on Ubuntu, look up the details described in **Appendix A**.

The Holy Grail for xv6

- [xv6-risc book](#)
- [xv6-risc src booklet](#)

The xv6 source tree is organised to mirror OS subsystems:

- **kernel/** → process management, memory, file system, traps
- **user/** → user programs
- **mkfs/** → file system creation tool

Each directory corresponds to a logical OS component, enabling systematic exploration and modification. **You will mostly interact with the **kernel/** and **user/** directory.**

Submission Guidelines

Create a patch with:

```
Shell  
➤ ./patch.sh
```

Rename the generated **lab8.patch** to **<rollno>_lab8.patch** and submit via Moodle.

The goal of this lab is to understand the kernel primitives and implementation details of library functions related to threading. In the xv6-public directory accompanying this lab, look for the comment with the text **WPTHREAD** (World Peace Threads) for a starting point to add implementation details for the tasks ahead.

0. trapped in trapframes

Threads are lightweight processes!

A unified approach to managing processes and threads in xv6 (and other OSes) is to treat both these execution instances as tasks and use the same PCB structure to store information about them. The information within the PCB serves the role of distinguishing threads from a process.

Threads have **per-thread** state — CPU registers (as part of their context), a user stack and a kernel stack. Further, threads of a thread group have a single address space, (can) file descriptors, signals, etc.

From a xv6 perspective, since threads share a common page table, the **trapframe** for each thread is mapped to the same user virtual address and the same physical address. This is an undesirable situation, as user and kernel runtime context is stored on the trapframe during interrupts and return from interrupts, and a single trapframe cannot be shared across instances of execution. As a result, we need to design **a per-thread trapframe virtual address region**.

One way to do this is using a trapframe page for each process table index, allowing for an easy mapping between thread/process and trapframe.

The following macro has been given to convert an index into a trapframe. This index can be the same as the index in the PCB table used to identify the PCB of a proc.

```
C/C++
#define TRAPFRAME(p) (TRAMPOLINE - ((p)+1)*PGSIZE)
```

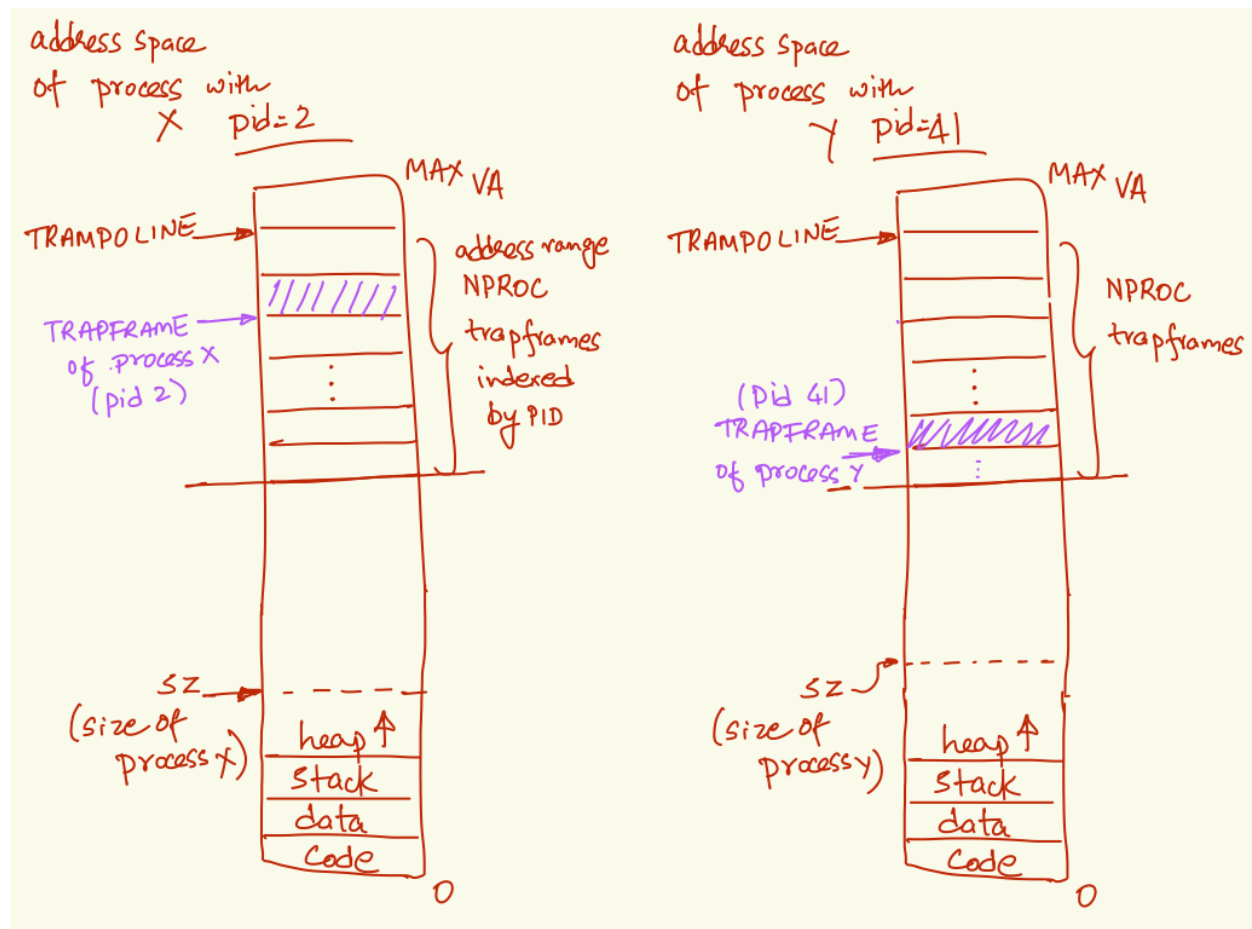
See **proc_pagetable** in **proc.c** for an example usage of this macro. In this case, the index of the PCB is calculated and passed to this macro to get its trapframe address.

TASK

As a first task, we will design a per-PCB trapframe within xv6, so that we can reuse it for threads in the next step.

All references to the macro have been updated. However, the current **trampoline.S**, which loads and stores the trapframe, depends on a fixed virtual address that is process-independent.

Update this file, as well as all other places where it is called from (**forkret** and **usertrap**), so that it is properly able to access the relevant (process-specific) trapframe.



The above figure is a representation of a per-process/PCB trapframe design. The address space below TRAMPOLINE is reserved for the NPROC number of trapframes. The trapframes can be assumed to be arranged in a linear order indexed by the index of the process in the PCB table/array. For example, process X at index 2 in the proc/PCB array uses the 2nd TRAPFRAME address to map its own trapframe and process Y at index 8 in the proc array uses the 8th TRAPFRAME address to map its trapframe.

Hints:

1. **sscratch** the Supervisor scratch register is a register that can be updated only while executing in the supervisor/kernel mode.
2. A design is to write to the **sscratch** register the address of the trapframe of the user process that the kernel is returning to. On an interrupt when the process is executing, the

usertrap handler can then look up the value in the **sscratch** register to find its trapframe address for saving user context and retrieving kernel state.

3. Note that for this, code at **usertrap** and **userret** labels of **trampoline.S** will have to be updated.
4. The instructions **csrr**, **csrw** and **csrrw** can be used to access **sscratch**.

```
csrr a0, sscratch      // read from sscratch in a0
csrw sscratch, a0      // write value in a0 to sscratch
csrrw a0, sscratch, a0 // swap values in a0 and sscratch
```

All CSR (control and status register) update instructions are atomic operations.

1. let's dolly sheep the processes

In this question, we will implement the **clone()** system call. This system call creates a new kernel thread for a process. Kernel threads are independently schedulable entities (processes), and all the threads of a process share the same virtual address space and file descriptors. Each of them has its own user stack in the address space. For this lab, we will assume that each thread's user stack is of a fixed size — one page (4096 bytes).

In order to allow for multi-threading in xv6, we need to come up with a design for the Thread Control Blocks (TCBs). For this lab, we will use **struct proc** itself as the PCB-cum-TCB for all the threads.

- **pid**: Same as process id and is synonymous with **thread_id**
- **tgid**: A thread-group-ID, which is the main process's (first calling process's) pid. All threads that belong to a process have the same **tgid**
- **thread_count**: number of threads of a process, only maintained for the main process
- **user_stack**: pointer to the user stack of the thread — each thread has its own user stack and kernel stack (similar to a process).

The thread-group-id and pid for the main thread would be the same value, and for the other spawned threads, the thread-group-id would be the same as the pid of the main thread. Threads other than the main thread do not have to bother with the **thread_count** field. **thread_count** is assigned the value of 1 for each process (refer to **allocproc** in **proc.c**), and the count increments when a clone call is made from the main thread process.

Note that our current implementation of clone is not supposed to support multiple levels of threading (i.e., a thread spawning threads), because of how we maintain the difference between the main and child threads.

The declaration of the clone system call is as follows:

```
int clone(void(*fn)(int *), int *arg, void *stack);
```

- **fn** : indicates the start point of execution of the thread.
- **arg** : the argument passed to the thread function **fn**, of type **int ***.
- **stack** : base address of the user stack allocated for the new thread.

Note that we are designing the system call to work only with functions that take one integer argument.

On success, the clone call returns the thread-id (pid) of the new thread or -1 on error.

A wrapper function is to be used in user space for invoking the clone system call —

```
int create_thread(void(*fn)(int*), int *arg);
```

- **fn** : indicates the start of execution of the thread
- **arg** : argument of function **fn** (hence, **int***)

Notice the distinction between arguments in **create_thread** and **clone**.

In this wrapper function, one page worth of memory in user space is allocated, and then the **clone** system call is called with the malloc'ed address as the stack argument to start the thread. On success, this call returns the **thread-id** of the new thread, and if unsuccessful, it returns -1.

The above step is required, since each thread has its own user space stack (or user stack). All the threads have a common address space and share the entire memory region, but operate using separate user stacks and per-thread state (maintained within the kernel).

An implementation of this function is given in the file **user/ulib.c**.

Following are the implementation details of the **clone** system call:

1. The stack pointer is passed from the thread wrapper function (which allocates memory in the virtual space of the calling process). This stack pointer (virtual address in the calling process, pointing to a single page) will be used as the user stack of the cloned thread.
2. Following are the expectations of the **clone** system call:
 - a. All the threads of a process have the same virtual address space and the same physical page mappings (and hence share a single page table).
 - b. Every thread has its own user stack for execution of userspace functions.
 - c. To initialise a thread, the instruction pointer and the stack pointer of the new thread need to be initialised.

- d. A new thread, once created and initialised, when scheduled, should start executing at the user-specified function.
- e. Each thread **MUST** end execution with an explicit call to **exit()**.
- f. If the thread calling **clone** is not the main thread, the call should fail.

Study the modification to the different system calls and functions in the **proc.c** source file.

Each thread has its own **sz** value; however, they share a pagetable. So **sbrk()** must update this for all threads in a process. Modify it to add this behaviour.

2: join the queue

As part of this task, implement the **join** system call that reaps an exited thread spawned by the process calling **join**. The declaration of the join system call is as follows: **int join(void);**

Like **wait**, **join** is a blocking system call that returns the thread-ID of a reaped thread when called from the main thread. If none of the threads has exited, then the call waits for one of the threads to exit. If there are no threads left to exit, then join will return -1.

Reaping a thread involves cleaning up its TCB(-cum-PCB), and updating the PCB of the process, which consists of steps like:

- Freeing up the kernel stack of the thread (we are not freeing the user stack, as explained later, on thread exit).
- Updating the TCB entries of the kernel thread to their respective default values, like in the implementation of **wait** (for example, set the state field to UNUSED).
- Decrementing the thread count of the process.
- Remember that threads share the same page table, so we must not deallocate the page table when a thread exits, since other threads might be accessing the same. Deallocation of the page table has to be done in **wait** when the process exits.
- A process should only be able to reap threads that it spawned, not the threads created by other processes. This should be checked when iterating over the proc table.

Based on our design, the join may lead to a page-sized memory leak due to the stack memory allocated earlier and not freed on thread termination. Our current design will keep things simple and let this be. As an extension, think about how to overcome these leaks cleanly.

The following parts of the implementation also have to be written:

1. **exit**: If a process exits, pass child processes to init, kill all the child threads and set them to UNUSED, before setting the state of the parent process to ZOMBIE.
2. **wait**: If a thread calls wait, do nothing and return -1. If process calls wait, look for exited child processes and reap them. Ignore child threads.

3. **fork**: If a thread calls fork, do nothing and return -1.

Note: The **create_thread** wrapper function should be available as part of every program that runs in xv6. Thus, we should add prototypes to user/user.h and the actual code to implement the library routines in user/ulib.c.

Use the test cases **tc-var.c** and **tc-array.c** to test the implementation.

tc-var.c

Spawns 20 threads, each of which increments a global counter by 1. If done right, the final value of VAR should be 20, but since we are trying to simulate a race condition here, we may not see the value 20 as output.

tc-array.c

Declares 2 arrays and then spawns 2 threads. The first thread sums the elements of the first half of the first array and the second half of the second array. The second thread sums up the remaining elements (second half of the first array, first half of the second array). The program then adds up these two values in the main function and prints the final sum.

THE OPTIONAL BOX

We are not freeing the user stack when a thread terminates. An easy way to do this is to free the malloc'ed user page when join is called. The problem is that join() is a system call implemented in proc.c whereas free() is a user function that cannot be called in kernel code. Figure out a way to get around this using wrapper functions, as done for clone.

Hint: **join()** can be modified to get the original stack pointer by storing it in the PCB, just as **wait()** can give you the exit status.

Appendix A

Follow these steps for installing xv6 on a Ubuntu-based Linux machine:

1. Prerequisites

- a. First, update your system:

sudo apt update

- b. Install the required packages:

**sudo apt install -y git build-essential qemu-system-riscv64
gcc-riscv64-unknown-elf**

- **build-essential**

For essential packages like libc, gcc, make, etc.

- **qemu-system-riscv64**

Emulates a RISC-V machine in software.

Your computer's real CPU speaks x86-64, but xv6 is written for RISC-V.

QEMU **pretends to be a RISC-V computer**, so the OS can run even though the hardware is different.

- **gcc-riscv64-unknown-elf**

It runs on your x86-64 machine but produces **RISC-V binaries**, not x86-64 ones.

We need this because xv6 must be compiled for the **CPU architecture that QEMU is emulating**.

2. Verify toolchain installation

riscv64-unknown-elf-gcc --version

qemu-system-riscv64 --version

If both commands work, your environment is ready.

3. Clone the xv6 RISC-V Repository

**git clone <https://github.com/mit-pdos/xv6-riscv.git>
cd xv6-riscv**

4. Build xv6

make

Run the command to build the OS. If compilation succeeds, you are ready to run xv6.

5. Run xv6 on QEMU

Start the xv6 operating system:

make qemu

Appendix B

A list of riscv ISA items and xv6 items that intersect with the xv6 system call implementation — For details of these and more, look up Chapter 4 of the xv6 book.

None

ecall

sret

stvec

sepc

scause

sstatus (SIE and SPP)

sscratch

satp

csrw

csrrw

a0

TRAMPOLINE

TRAPFRAME

uservec

usertrap

userret

prepare_return

syscall

copyin

copyout

copyinstr

argint

argstr